School of Computer Science and Artificial Intelligence

Lab Assignment - 12.3

Course Name : AI Assistant Coding
Name of the Student : Shaik Naved Ahmed
Registration No : 2303A54053
Batch No : 47-B

Task 1: Sorting Student Records for Placement Drive

Scenario SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:

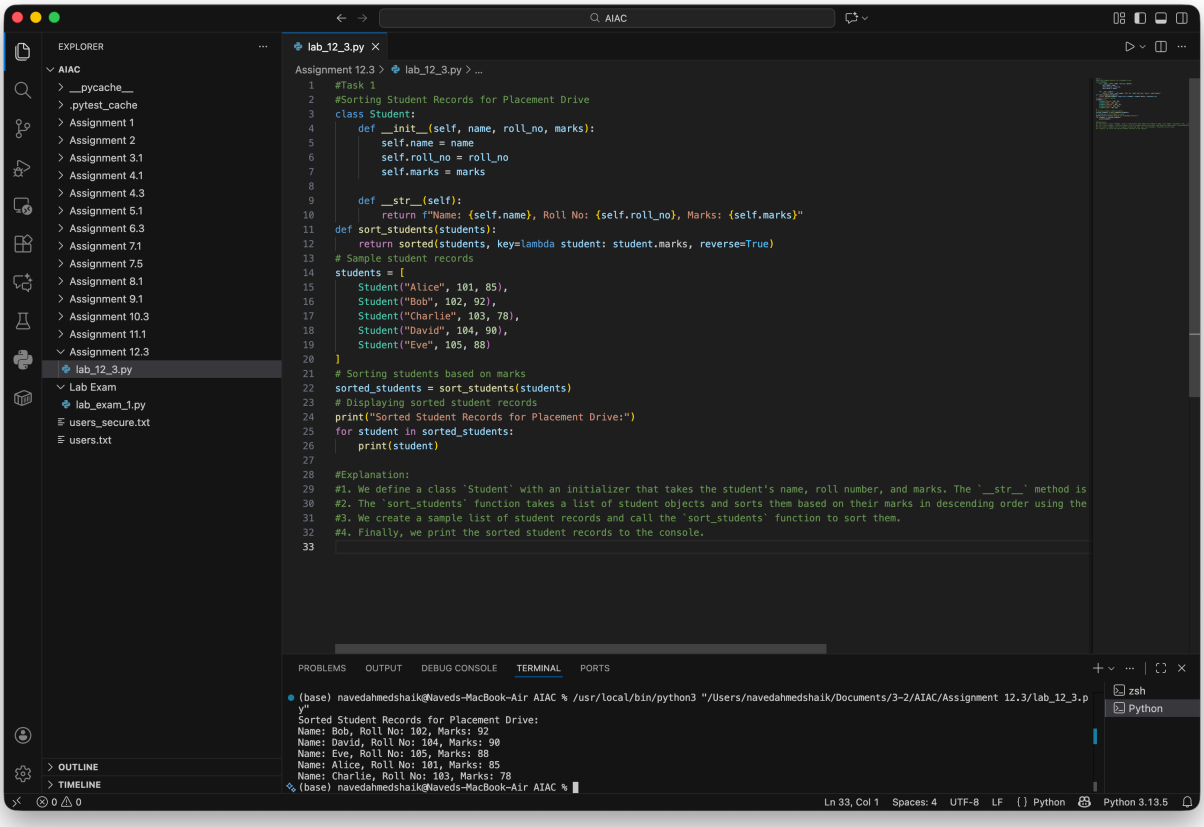
o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

• Correctly sorted student records.
• Performance comparison between Quick Sort and Merge Sort.
• Clear output of top-performing students.

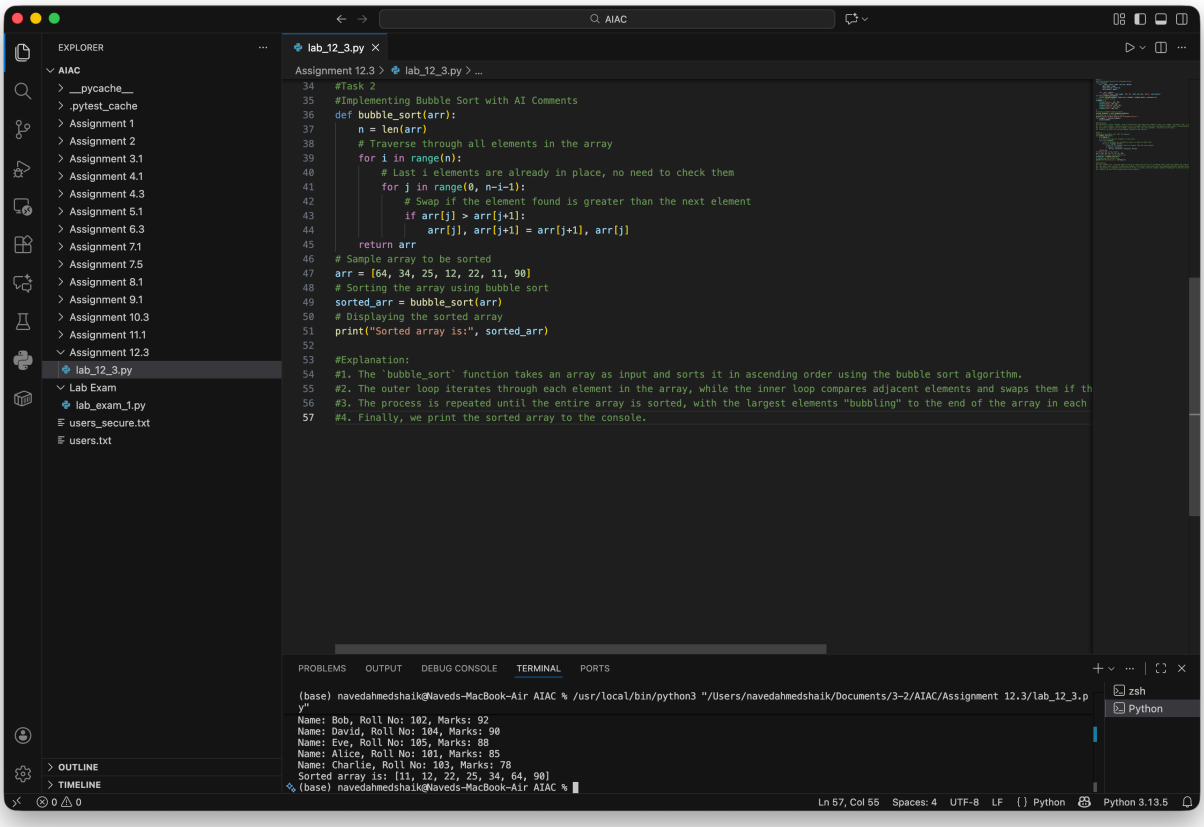


Explanation:

1. We define a class `Student` with an initializer that takes the student's name, roll number, and marks. The `__str__` method is overridden to provide a string representation of the student object.
2. The `sort_students` function takes a list of student objects and sorts them based on their marks in descending order using the `sorted` function and a lambda function as the key.
3. We create a sample list of student records and call the `sort_students` function to sort them.
4. Finally, we print the sorted student records to the console.

Task 2: Implementing Bubble Sort with AI Comments

• Task: Write a Python implementation of Bubble Sort.
• Instructions:
• Students implement Bubble Sort normally.
• Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
• Request AI to provide time complexity analysis.
• Expected Output:
• A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

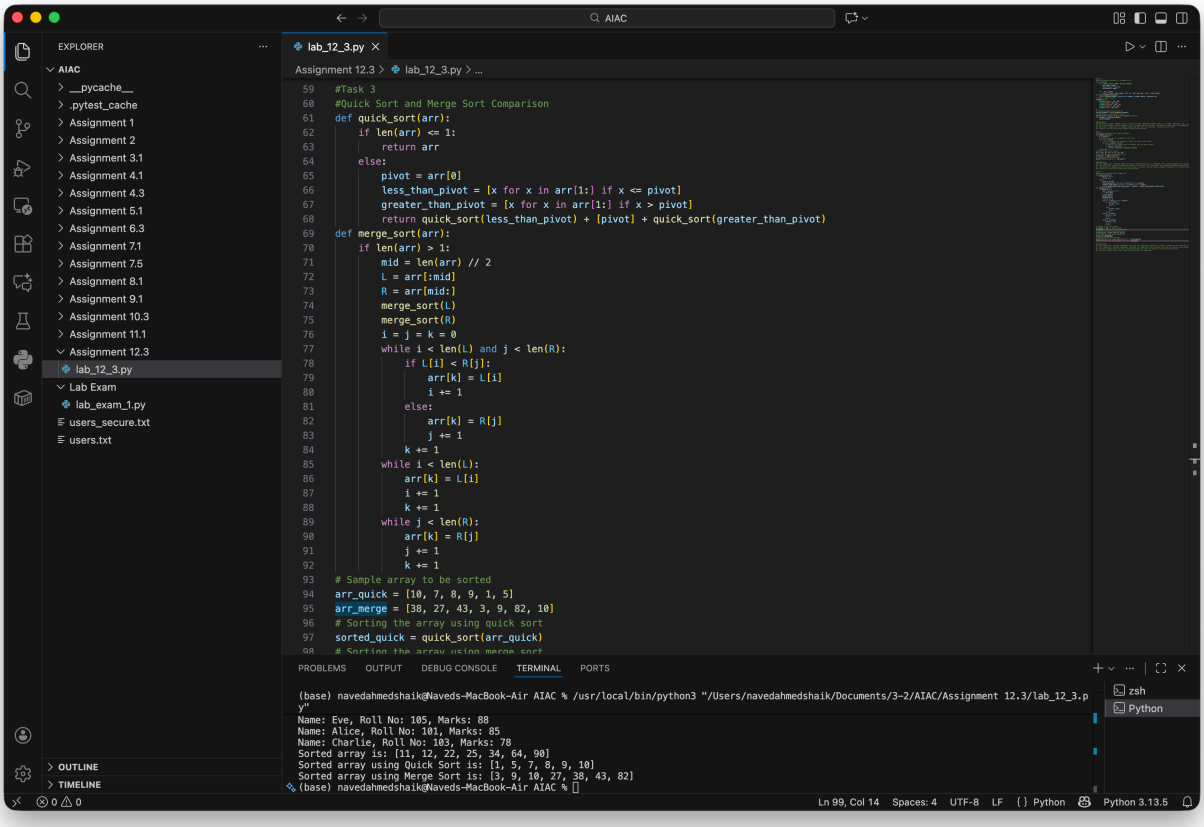


Explanation:

1. The `bubble_sort` function takes an array as input and sorts it in ascending order using the bubble sort algorithm.
2. The outer loop iterates through each element in the array, while the inner loop compares adjacent elements and swaps them if they are in the wrong order.
3. The process is repeated until the entire array is sorted, with the largest elements "bubbling" to the end of the array in each iteration.
4. Finally, we print the sorted array to the console.

Task 3: Quick Sort and Merge Sort Comparison

• Task: Implement Quick Sort and Merge Sort using recursion.
• Instructions:
• Provide AI with partially completed functions for recursion.
• Ask AI to complete the missing logic and add docstrings.
• Compare both algorithms on random, sorted, and reverse-sorted lists.
• Expected Output:
• Working Quick Sort and Merge Sort implementations.
• AI-generated explanation of average, best, and worst-case complexities.



Explanation:

1. The `quick_sort` function implements the quick sort algorithm, which uses a pivot to partition the array into two sub-arrays and recursively sorts them.
2. The `merge_sort` function implements the merge sort algorithm, which divides the array into halves, recursively sorts them, and then merges the sorted halves back together.
3. We create sample arrays for both sorting algorithms and call the respective functions to sort them.
4. Finally, we print the sorted arrays to the console for comparison.

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

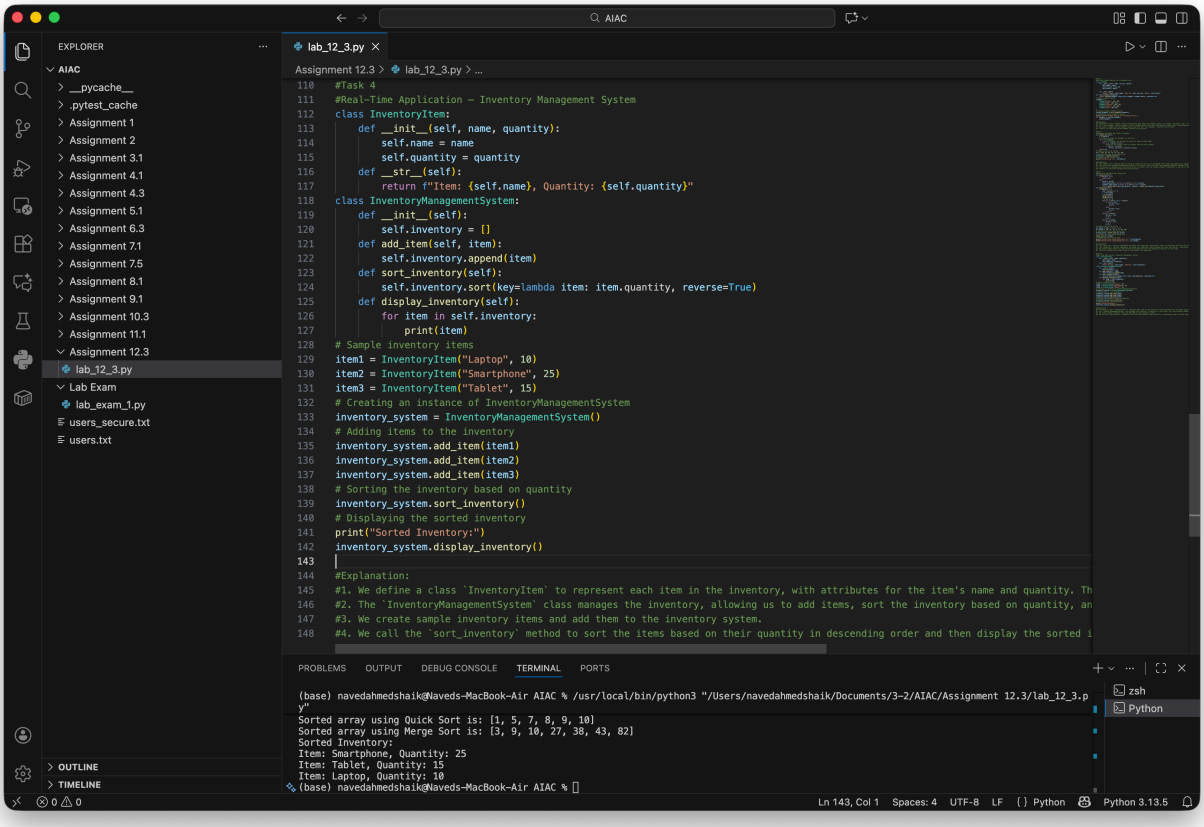
1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.



Explanation:

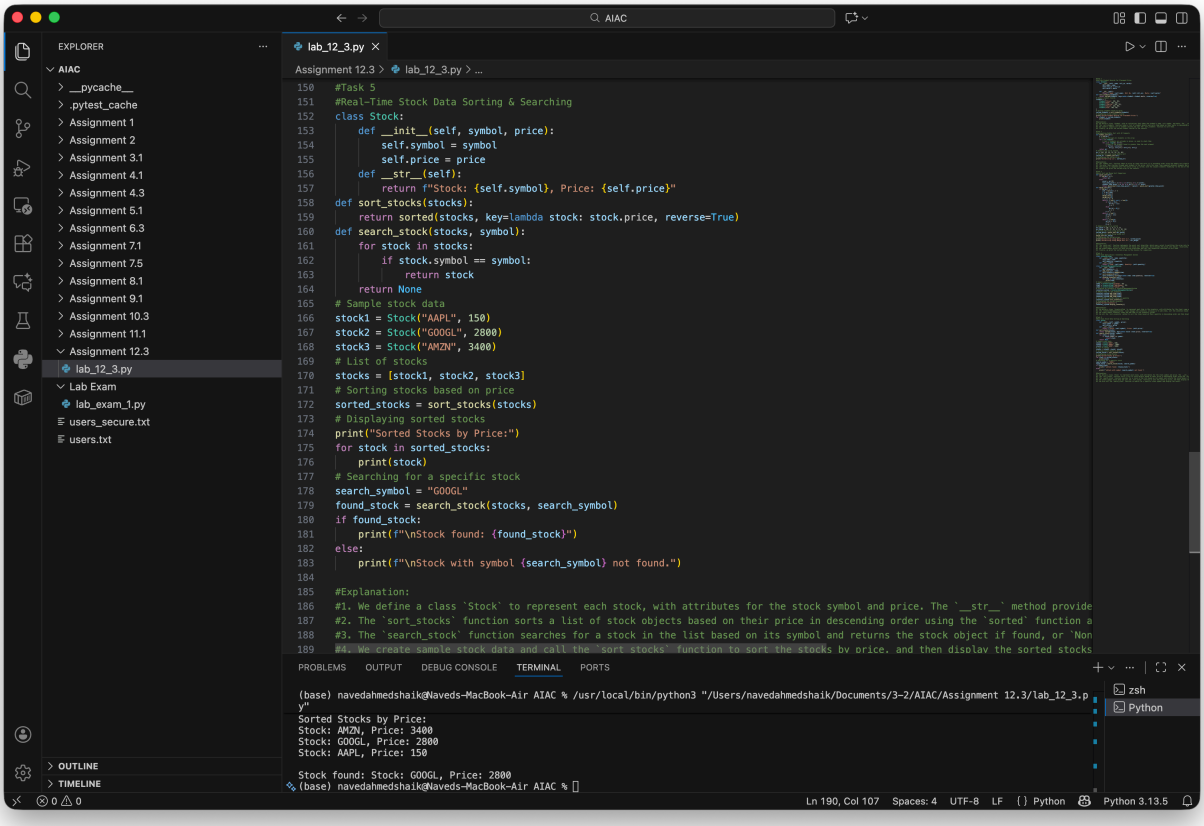
1. We define a class `InventoryItem` to represent each item in the inventory, with attributes for the item's name and quantity. The `__str__` method provides a string representation of the item.
2. The `InventoryManagementSystem` class manages the inventory, allowing us to add items, sort the inventory based on quantity, and display the inventory.
3. We create sample inventory items and add them to the inventory system.
4. We call the `sort_inventory` method to sort the items based on their quantity in descending order and then display the sorted inventory

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by percentage change.
- Implement a search function that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with Heap Sort and searching with Hash Maps.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.



Explanation:

1. We define a class `Stock` to represent each stock, with attributes for the stock symbol and price. The `__str__` method provides a string representation of the stock.
2. The `sort_stocks` function sorts a list of stock objects based on their price in descending order using the `sorted` function and a lambda function as the key.
3. The `search_stock` function searches for a stock in the list based on its symbol and returns the stock object if found, or `None` if not found.
4. We create sample stock data and call the `sort_stocks` function to sort the stocks by price, and then display the sorted stocks.
5. We also call the `search_stock` function to search for a specific stock symbol and display the result.